UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

CENTRO UNIVERSITARIO DE OCCIDENTE

DIVISIÓN CIENCIAS DE LA INGENIERÍA

ORGANIZACIÓN DE LENGUAJES Y COMPILADORES 1

CATEDRÁTICO: ING. JOSE MOISES GRANADOS GUEVARA

MANUAL TECNICO
PRACTICA 2

PABLO DANIEL ALVARADO RODRIGUEZ

CARNÉ: 202130534

QUETZALTENANGO, ABRIL 2026

Introducción

El siguiente documento ofrece al desarrollador detalles importantes sobre la arquitectura y el método de construcción de un sistema de creación de analizadores sintácticos utilizando la herramienta de Jison, esto por el lado del Frontend. Yéndonos por el lado del backend tendremos el diagrama de clases de una API la cual permite la persistencia de datos y que es clave para el funcionamiento de la APP.

Se espera que esta documentación sea suficiente para que el desarrollador/programador entienda la base de cómo funciona el sistema.

Índice

Tecnologías Utilizadas....4
Front-end....4
Back-end....4
Hardware....5
Software....5
Análisis de Gramática para el Analizador Léxico....5
Análisis de Gramática para el Analizador Sintáctico....7
Generación de Gramáticas....14
Calcular Primeros....14
Calcular Siguientes....16
Diagrama de Clases....18
Diagrama de clases del Front-end....18
Diagrama de clases del Back-end....18

Tecnologías Utilizadas

Front-end

- **Svelte:** Se utilizó este Framework para la realización de la app web ya que permite la inserción de código en tiempo real sin la necesidad de realizar un re-deploy a la app. Para este proyecto se utilizó la versión 5.55.1.

- **HTML:** Este es un lenguaje de marcado que permite crear la estructura de los sitios web, se utilizó la versión 5 de este lenguaje.

- **JavaScript:** Lenguaje de programación que permite desarrollar todas las acciones e interacciones de la app.

- **SCSS:** Se utilizó para darle estilos a la app y que fuera más amigable y atractiva para el usuario.

- **Node.Js:** Se utilizó esta tecnología para el backend interno de la aplicación. Permite la interacción entre la app y diferentes programas instalados en el sistema como Jison y otras librerías que necesitan nativamente el framework. Se utilizó la versión 22.20.0 de esta tecnología.

- **Jison:** Este es un LALR1 que nos permite la generación de gramáticas y también puede ser utilizado para la generación de analizadores sintácticos.
  En el caso de nuestro proyecto nos ayuda a generar analizadores descendentes LL1.
  Para este Proyecto se utilizó la versión 0.4.18.

Back-end

- **Java:** Este es un lenguaje de programación fuertemente tipado y que utiliza el paradigma de Programación Orientada a Objetos.
  Este lenguaje nos permitió crear una API para poder comunicar la aplicación WEB con una base de datos.
  En este proyecto se utilizó la versión 21 de Java y la versión 21 del JDK.

- **MySQL:** Este es un motor de base de datos que usa el lenguaje SQL para la manipulacion de datos. Para este proyecto se utilizó la versión Community de este software que es 8.4.7.

Hardware

Las especificaciones del hardware utilizado son las siguientes

- **Sistema Operativo:** Se utilizó Ultramarin-Linux que es un sistema con base Fedora. La versión de la base Fedora fue Fedora 41.

- **Kernel:** Se utilizó el kernel 6.17.10-100 en el sistema operativo.

- **RAM:** Para la realización de este proyecto se utilizó una máquina con 16GB de memoria RAM debido a que se requería tener dos aplicaciones encendidas al mismo tiempo.

- **Almacenamiento:** La máquina utilizada cuenta con 240GB de almacenamiento utilizando un Disco de Estado Sólido (SSD).

Software

Para la realización de esta aplicación se utilizaron diferentes programas de software, los cuales por comodidad se recomienda.

- **Visual Studio Code (VSCode):** Utilizado para el desarrollo de la parte frontend.
- **Netbeans:** Utilizado para el desarrollo de la parte del backend.

Análisis de Gramática para el Analizador Léxico

Para el analizador Léxico utilizamos las bondades que la librería Jison nos ofrece y por tanto a continuación se mostraran las expresiones regulares que se utilizaron para el lexer implementado.

```
%lex
%options ranges yylineno
%%

//* ignorar espacios en blanco
\s+                    /* ignorar espacios en blanco */
```

```
[\u200B\u200C\u200D\uFEFF\u00A0]+ /* ignorar caracteres invisibles y
nbsp */
"#".*                    /* ignorar comentarios de linea */
"/**"[\s\S]*?"*/"        /* ignorar comentarios de bloque */

//* palabras reservadas

"Wison"                   return 'WISON';

"Lex"                     return 'INICIO_LEX';

"Syntax"                  return 'INICIO_SYNTAX';

"Terminal"                return 'TERMINAL';
"No_Terminal"             return 'NO_TERMINAL';
"Initial_Sim"             return 'INITIAL_SIM';

//* Simbolos

"_"                      return 'GUION_BAJO';
"?"                       return 'QUESTION_UP';
"¿"                      return 'QUESTION_DOWN';
"{"                       return 'LLAVE_ABIERTA';
":"                       return 'DOS_PUNTOS';
"}"                       return 'LLAVE_CERRADA';
"<"                      return 'MENOR';
"-"                      return 'GUION';
"="                       return 'ASIG_LEX';
";"                      return 'PUNTO_Y_COMA';

//* Identificadores

"$_"[a-zA-Z0-9_]+        return 'ID_TERMINAL';
"%_"[a-zA-Z0-9_]+        return 'ID_NO_TERMINAL';

//* Literales
\'[^\\']*\'               { yytext = yytext.slice(1, -1); return
'CADENA';}
\[aA\-zZ\]               return 'RANGO_LETRAS';
\[0\-9\]                 return 'NUMERO';


//* Operadores
```

```
"*"                     return 'ESTRELLA_KLEENE';
"+"                     return 'CERRADURA_POSITIVA';
"|"                     return 'ALTERNACION';
"("                     return 'PARENTESIS_ABIERTO';
")"                     return 'PARENTESIS_CERRADO';

//* Fin de archivo

<<EOF>>                 return 'EOF';
.                       {
                          if (yy && typeof yy.registrarErrorLexico ===
'function') {
                              yy.registrarErrorLexico(yytext, yylineno +
1, yylloc.first_column + 1);
                          }
                      }

/lex
```

Análisis de Gramática para el Analizador Sintáctico

Nuevamente Jison nos permite utilizarlo para generar nuestro analizador sintáctico permitiendo la declaración de terminales y no terminales, asimismo permite la estructuración de reglas de producción que nuestro lenguaje requiere.

```
/*
* Analizador Sintactico
*/

//* Produccion inicial o raiz
%start programa_wison

%% //* Inicio de la gramatica
```

```
//* Produccion raiz
programa_wison
   : inicio_wison bloque_lex bloque_syntax fin_wison EOF
   { $$ = { lex: $2, syntax: $3, errors: { lexical: erroresLexicos,
syntactic: erroresSintacticos } }; return $$; }
   | error fin_wison EOF
   {
       registrarErrorSintactico(
           'La estructura del archivo es inválida.',
           yytext,
           @1.first_line,
           @1.first_column + 1
       );
       $$ = { lex: null, syntax: null, errors: { lexical:
erroresLexicos, syntactic: erroresSintacticos } };
   }
   ;

inicio_wison
   : WISON QUESTION_DOWN
   ;

fin_wison
   : QUESTION_UP WISON
   ;

//* Bloque Lexico
bloque_lex
   : inicio_lex lista_declaraciones_lex fin_lex
   { $$ = { terminals: $2 }; }
   | error fin_lex
   {
       registrarErrorSintactico(
           'El bloque léxico está incompleto o mal formado.',
           yytext,
           @1.first_line,
           @1.first_column + 1
       );
       $$ = { terminals: [] };
   }
   ;
```

```
inicio_lex
    : INICIO_LEX LLAVE_ABIERTA DOS_PUNTOS
    ;

fin_lex
    : DOS_PUNTOS LLAVE_CERRADA
    ;

//* lista de declaraciones lexicas
lista_declaraciones_lex
    : lista_declaraciones_lex declaracion_terminal
    { $$ = $1.concat([$2]); }
    | declaracion_terminal
    { $$ = [$1]; }
    ;

//* Declaracion de terminales
declaracion_terminal
    : TERMINAL ID_TERMINAL asig_lex expresion_lexica PUNTO_Y_COMA
    { $$ = { kind: 'terminal', id: $2, expr: $4 }; }
    | error PUNTO_Y_COMA
    {
        registrarErrorSintactico(
            'Declaración de terminal incompleta o mal formada.',
            yytext,
            @1.first_line,
            @1.first_column + 1
        );
        $$ = { kind: 'terminal', id: null, expr: [] };
    }
    ;

asig_lex
    : MENOR GUION
    ;

//* Expresiones Lexicas
expresion_lexica
    : lista_elementos_lexicos
    { $$ = $1; }
    ;

lista_elementos_lexicos
```

```
    : lista_elementos_lexicos elemento_lexico
    { $$ = $1.concat([$2]); }
    | elemento_lexico
    { $$ = [$1]; }
    ;

elemento_lexico
    : unidad_lexica operador_unario
    { $$ = { type: 'unario', value: $1, operator: $2 }; }
    | unidad_lexica
    { $$ = $1; }
    ;

operador_unario
    : ESTRELLA_KLEENE
    { $$ = '*'; }
    | CERRADURA_POSITIVA
    { $$ = '+'; }
    | QUESTION_UP
    { $$ = '?'; }
    ;

unidad_lexica
    : CADENA
    { $$ = { type: 'literal', value: $1 }; }
    | RANGO_LETRAS
    { $$ = { type: 'alfanumerico', value: $1 }; }
    | NUMERO
    { $$ = { type: 'numero', value: $1 }; }
    | ID_TERMINAL
    { $$ = { type: 'terminal_ref', value: $1 }; }
    | PARENTESIS_ABIERTO lista_elementos_lexicos PARENTESIS_CERRADO
    { $$ = $2; }
    ;

//* Bloque Sintactico
bloque_syntax
    : inicio_syntax lista_no_terminales declaracion_inicial
lista_producciones fin_syntax
    { $$ = { nonTerminals: $2, startSymbol: $3, productions: $4 }; }
    | error fin_syntax
    {
        registrarErrorSintactico(
```

```
            'El bloque sintáctico está incompleto o mal formado.',
            yytext,
            @1.first_line,
            @1.first_column + 1
        );
        $$ = { nonTerminals: [], startSymbol: null, productions: [] };
    }
    ;

inicio_syntax
    : INICIO_SYNTAX LLAVE_ABIERTA LLAVE_ABIERTA DOS_PUNTOS
    ;

fin_syntax
    : DOS_PUNTOS LLAVE_CERRADA LLAVE_CERRADA
    ;

//* Declaracion de no terminales
lista_no_terminales
    : lista_no_terminales declaracion_no_terminal
    { $$ = $1.concat([$2]); }
    | declaracion_no_terminal
    { $$ = [$1]; }
    ;

declaracion_no_terminal
    : NO_TERMINAL ID_NO_TERMINAL PUNTO_Y_COMA
    { $$ = $2; }
    | error PUNTO_Y_COMA
    {
        registrarErrorSintactico(
            'Declaración de no terminal incompleta o mal formada.',
            yytext,
            @1.first_line,
            @1.first_column + 1
        );
        $$ = null;
    }
    ;

//* Declaracion del simbolo inicial
declaracion_inicial
    : INITIAL_SIM ID_NO_TERMINAL PUNTO_Y_COMA
```

```
    { $$ = $2; }
    | INITIAL_SIM error PUNTO_Y_COMA
    {
        registrarErrorSintactico(
            'Declaración del símbolo inicial incompleta o mal formada.',
            yytext,
            @2.first_line,
            @2.first_column + 1
        );
        $$ = null;
    }
    ;

//* Lista de producciones
lista_producciones
    : lista_producciones regla_produccion
    { $$ = $1.concat([$2]); }
        | lista_producciones error PUNTO_Y_COMA
        {
        registrarErrorSintactico(
            'Producción inválida. Omitida hasta encontrar un punto y
coma.',
            yytext,
            @2.first_line,
            @2.first_column + 1
        );
        $$ = $1;
        }
    | regla_produccion
    { $$ = [$1]; }
    ;

regla_produccion
    : ID_NO_TERMINAL asig_syn opciones_produccion PUNTO_Y_COMA
    { $$ = { lhs: $1, rhs: $3 }; }
    | ID_NO_TERMINAL asig_syn PUNTO_Y_COMA
    { $$ = { lhs: $1, rhs: [[]] }; }
    | ID_NO_TERMINAL error PUNTO_Y_COMA
    {
        registrarErrorSintactico(
            'Producción incompleta o mal formada.',
            yytext,
            @2.first_line,
```

```
            @2.first_column + 1
        );
        $$ = { lhs: $1, rhs: [] };
    }
    ;

asig_syn
    : MENOR ASIG_LEX
    ;

//* Regla de produccion

//* La parte derecha de la produccion puede ser una secuencia de
simbolos o varias opciones separadas por el operador de alternacion '|'
opciones_produccion
    : opciones_produccion ALTERNACION secuencia_simbolos
    { $$ = $1.concat([$3]); }
        | opciones_produccion ALTERNACION error
        {
        registrarErrorSintactico(
            'Una alternativa de la producción está incompleta.',
            yytext,
            @3.first_line,
            @3.first_column + 1
        );
        $$ = $1.concat([[]]);
        }
    | secuencia_simbolos
    { $$ = [$1]; }
    ;

//* Una secuencia de simbolos puede ser una lista de simbolos
terminales o no terminales, o puede ser vacia
secuencia_simbolos
    : secuencia_simbolos simbolo
    { $$ = $1.concat([$2]); }
    | simbolo
    { $$ = [$1]; }
    ;

//* Simbolos basicos

simbolo
```

```
    : ID_TERMINAL
    { $$ = { kind: 'idTerminal', value: $1 }; }
    | ID_NO_TERMINAL
    { $$ = { kind: 'idNonTerminal', value: $1 }; }
    ;
```

Generación de Gramáticas

Dada la naturaleza de la aplicación no hay una generación de gramáticas definida estrictamente. Por esta razón no es posible mostrar la implementación en código JavaScript para la generación de gramáticas.
Sin embargo sí es posible mostrar el algoritmo que se implementó para su realización y que de esta manera pueda ser replicado y entendido por el desarrollador.

Una vez se calculan los primeros y siguientes se procede a la creación de la tabla LL1. Consiste en recorrer cada no terminal y las producciones que este puede generar.

Calcular Primeros

La teoría indica que este es el conjunto de los tokens terminales que pueden aparecer al inicio de las cadenas derivadas desde α.

```
FUNCION agregarTodos(destino, origen) INICIO
   huboCambios <- false
   PARA CADA x DE origen INICIO
       SI (x NO ESTA EN destino) ENTONCES
           destino.add(x)
           huboCambios <- true
       FINSI
   FINPARA
   RETORNAR huboCambios
FIN

FUNCION obtenerPrimerosDeSecuencia(secuencia) INICIO
   resultado <- Set()
```

```
SI (secuencia.estaVacia()) ENTONCES
    resultado.add('LAMBDA')
    RETORNAR resultado
FINSI

todosPuedenSerLambda <- true

PARA CADA simbolo DE secuencia INICIO
    SI (simbolo.tipo == 'TERMINAL') ENTONCES
        resultado.add(simbolo.valor)
        todosPuedenSerLambda <- false
        ROMPER
    FINSI

    SI (simbolo.tipo == 'LAMBDA') ENTONCES
        resultado.add('LAMBDA')
        todosPuedenSerLambda <- false
        ROMPER
    FINSI

    SI (simbolo.tipo == 'NO_TERMINAL') ENTONCES
        primerosHijo <- primeros.get(simbolo.valor) O Set()
        hijoTieneLambda <- false

        PARA CADA token DE primerosHijo INICIO
            SI (token == 'LAMBDA') ENTONCES
                hijoTieneLambda <- true
            SINO
                resultado.add(token)
            FINSI
        FINPARA

        SI (hijoTieneLambda == false) ENTONCES
            todosPuedenSerLambda <- false
            ROMPER
        FINSI
    FINSI
FINPARA

SI (todosPuedenSerLambda == true) ENTONCES
    resultado.add('LAMBDA')
FINSI
```

```
   RETORNAR resultado
FIN
calcularPrimeros() INICIO
   huboCambios <- true

   MIENTRAS (huboCambios == true) HACER
       huboCambios <- false

       PARA CADA instruccion DE bloqueSintactico INICIO
           SI (instruccion.tipo != 'PRODUCCION') ENTONCES
               CONTINUAR
           FINSI

           noTerminal <- instruccion.padre
           setActual <- primeros.get(noTerminal)
           longitudAntes <- setActual.size

           PARA CADA alternativa DE instruccion.alternativas INICIO
               primerosAlt <- obtenerPrimerosDeSecuencia(alternativa)
               agregarTodos(setActual, primerosAlt)
           FINPARA

           SI (setActual.size > longitudAntes) ENTONCES
               huboCambios <- true
           FINSI
       FINPARA
   FINMIENTRAS
FIN
```

Calcular Siguientes

Este es el conjunto de tokens terminales que aparecen inmediatamente a la derecha del no terminal en una derivación.

```
calcularSiguientes(simboloInicial) INICIO
   siguientes.get(simboloInicial).add('$')
   huboCambios <- true

   MIENTRAS (huboCambios == true) HACER
       huboCambios <- false
```

```
        PARA CADA instruccion DE bloqueSintactico INICIO
            SI (instruccion.tipo != 'PRODUCCION') ENTONCES
                CONTINUAR
            FINSI

            A <- instruccion.padre

            PARA CADA alternativa DE instruccion.alternativas INICIO
                PARA i <- 0 HASTA alternativa.length - 1 INICIO
                    B <- alternativa[i]

                    SI (B.tipo != 'NO_TERMINAL') ENTONCES
                        CONTINUAR
                    FINSI

                    beta <- alternativa.slice(i + 1)
                    primerosBeta <- obtenerPrimerosDeSecuencia(beta)
                    followB <- siguientes.get(B.valor)

                    longitudAntes <- followB.size

                    PARA CADA t DE primerosBeta INICIO
                        SI (t != 'LAMBDA') ENTONCES
                            followB.add(t)
                        FINSI
                    FINPARA

                    SI (beta.estaVacia() ||
primerosBeta.tiene('LAMBDA')) ENTONCES
                        followA <- siguientes.get(A)
                        agregarTodos(followB, followA)
                    FINSI

                    SI (followB.size > longitudAntes) ENTONCES
                        huboCambios <- true
                    FINSI
                FINPARA
            FINPARA
        FINPARA
    FINMIENTRAS
FIN
```

Diagrama de Clases

Diagrama de clases del Front-end

Para representar cómo están relacionados los diferentes archivos del front-end se realizó una representación visual en la app web de draw.io para su óptima visualización.

https://app.diagrams.net/#G1OPRBFJspL0rKliqySQ2e_6OZfB9xMCmz#%7B%22pageId%22%3A%22Q9t4qzC9opTRNSho73pO%22%7D

Diagrama de clases del Back-end

Para poder visualizar la forma en que la API se ha estructurado se deja un enlace al diagrama de clases que la componen.

https://app.diagrams.net/#G1s_d7gYXWaN1q7hya8ksZ01QDCbxxO-oy#%7B%22pageId%22%3A%22GZ24EjME-lolAAJxVbtY%22%7D